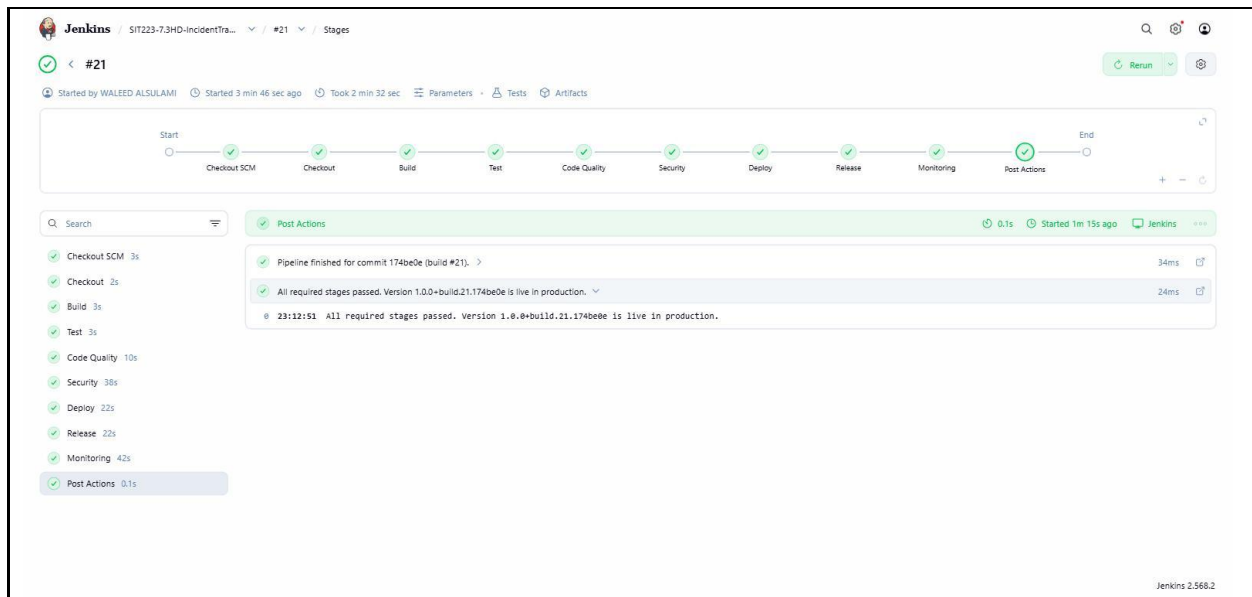


SIT223/753 – HD Task - Answer Sheet

1. A link to the demo video [PENDING – VIDEO LINK] Insert the real demo video URL here after recording. See README.md (section “Handover”) in the repository for the exact steps to replace this marker and re-export this PDF.
2. Provide a link to your GitHub repository containing the Jenkins pipeline script. Please ensure that your marking tutor has been granted appropriate access https://github.com/8ccs/sit223-7.3hd-incident-tracker-new2 This repository is public, so both the Marker and the Unit Chair already have full read access to the code and the Jenkinsfile without needing an individual invite. (The assignment brief requires access for both roles, even though this template section only names the marking tutor.)
3. Specify how many stages you have implemented 7 of 7 stages implemented and verified end-to-end: Build, Test, Code Quality, Security, Deploy, Release, and Monitoring.
4. A brief description of your project and the technologies used. Incident Tracker is a small but functionally deep incident-management web application, built as the subject of this seven-stage Jenkins pipeline. It provides a REST API for creating, reading, updating, and deleting incidents, plus status/severity filtering and free-text search, and a separate server-rendered dashboard that lists and filters incidents (the dashboard itself only displays and filters -- create/update/delete go through the API). Each incident has a title, description, severity (low/medium/high/critical), and status (open/in_progress/resolved/closed). A business rule blocks deletion of any incident that is still open or in progress. Data is stored in SQLite, with a separate database file for each environment. The app also exposes /health (readiness) and /metrics (Prometheus format) endpoints, which the Monitoring stage uses. Technologies: Python 3.11, Flask 3.1, prometheus-client, SQLite (standard library), and Waitress as the production WSGI server. Testing: pytest and pytest-cov. Code quality: flake8, pylint, radon. Security: Bandit and pip-audit. Monitoring: Prometheus and Alertmanager. CI/CD: Jenkins 2.568 (declarative pipeline), running natively on Windows. This machine did not have Docker installed, so the Build stage produces a versioned zip artifact instead of a container image, and Deploy/Release run the app as isolated per-environment Python processes instead of containers. This decision, and the reasoning behind it, is explained in the repository README.
5. A screenshot of your Jenkins pipeline. Jenkins Stage View, build #21, commit 174be0e -- all 7 required stages (Build, Test, Code Quality, Security, Deploy, Release, Monitoring) passed. Version 1.0.0+build.21.174be0e is live in production:



6. A brief description of each stage of your pipeline including the used frameworks/tools.

6.1. Build:

Jenkins runs `scripts/package_artifact.ps1`, which packages the app into a zip artifact and stamps its filename with the Git commit hash and the Jenkins build number (this run: `incident-tracker-1.0.0+build.21.174be0e.zip`). The artifact is archived by Jenkins and also copied to a shared artifact store, so every later stage uses the exact same file. The pipeline is triggered automatically by SCM polling every 5 minutes, because this local Jenkins has no public URL for a GitHub webhook to reach.

6.2. Test:

pytest runs 78 unit and integration tests: unit tests check validation rules, the SQLite data layer, and the security/quality gate scripts in isolation; integration tests call the real Flask routes, including failure cases such as invalid input and deleting an incident that is still open. Coverage is measured with `pytest-cov` and gated at 80% (currently 98%). Results are published to Jenkins through the JUnit plugin. A failing test, or coverage below the threshold, fails the build and stops Deploy and Release from running.

6.3. Code Quality

flake8, pylint, and radon run against the app code. `scripts/quality_gate.py` enforces two explicit thresholds: a pylint score of at least 8.0/10 (currently 9.96/10) and cyclomatic complexity no worse than rank C. This gate caught two real issues during development – a variable-scope bug and an overly complex function – both fixed; details are in the project README.

6.4. Security

Bandit scans the code for common vulnerability patterns, and `pip-audit` checks all 9 runtime dependencies against known CVEs (currently zero found). `scripts/security_gate.py` blocks the build on any Bandit finding at HIGH severity with MEDIUM-or-higher confidence, and on any dependency vulnerability that has an available fixed version; it also fails if a scan report is missing, malformed, or reflects a scan that never actually completed, rather than only checking for findings (a real bug in the gate itself, found and fixed during this review). Two real Bandit findings were fixed during development -- B104 (binding to all network interfaces) and one of two B608 (dynamic SQL) findings, rewritten to use fixed query text -- though both are MEDIUM severity

in Bandit's own classification, below this gate's HIGH-only blocking threshold either way. The second B608 finding, in the incident-update query, is also MEDIUM severity and is additionally mitigated: it needs a dynamic column list for partial updates, restricted to a fixed five-field allow-list checked in code before the query runs, with every value still sent as a parameter, never concatenated. The full findings table is in README.md section 7.1.

6.5. Deployment

scripts/deploy.ps1 extracts the build artifact into an isolated staging environment (its own port, virtual environment, and SQLite database), waits for a /health readiness check, then runs 5 real HTTP smoke tests against the live staging app before Release is allowed to run.

6.6. Release

The same artifact, never rebuilt, is deployed to a separate production environment by the same script, then health-checked again; the released version is recorded only after that health check passes, so a failed deploy never overwrites a good release record.

scripts/rollback.ps1 can redeploy the previous version with no rebuild, and is exposed as a Jenkins boolean parameter (ROLLBACK_PRODUCTION). This was verified directly: deploy one version, release a second, simulate an incident and recover (which restarts the currently live version), then roll back, and confirm the app returns to the first version.

6.7. Monitoring

Prometheus scrapes live /metrics from the production app every 5 seconds. Three alert rules watch for the app being down, a high error rate, and high latency. Alertmanager routes every firing alert to two destinations at once: a local webhook inbox (used for automated verification) and a real Slack channel via Alertmanager's own slack_configs receiver, not a generic webhook handler. scripts/verify_alert_path.py automates the full path on every pipeline run: it stops production, confirms Prometheus marks it down, confirms the alert fires, confirms the notification arrives, restarts production on the same version, then confirms the target recovers, the alert clears, and a resolved notification arrives. Recovery is always attempted, even if the check itself fails partway through, and the build still fails if verification failed even though recovery succeeded -- verified end-to-end, including a deliberately failed run. Real Slack delivery was also verified directly with a real Incoming Webhook connected to a channel I control: a controlled incident produced one firing message and one resolved message, both confirmed actually present in the Slack channel, and confirmed again independently through Alertmanager's own notification-count metrics (2 successful Slack sends, 0 new failures, right after the webhook was connected).